AI Evaluation

Xây hệ thống evaluation cho sản phẩm AI: dataset, trace analysis, code evals, LLM judge, human review

Instructor: Mai Anh Nguyen (Blue)



INSTRUCTOR

Mai Anh Nguyen (Blue)

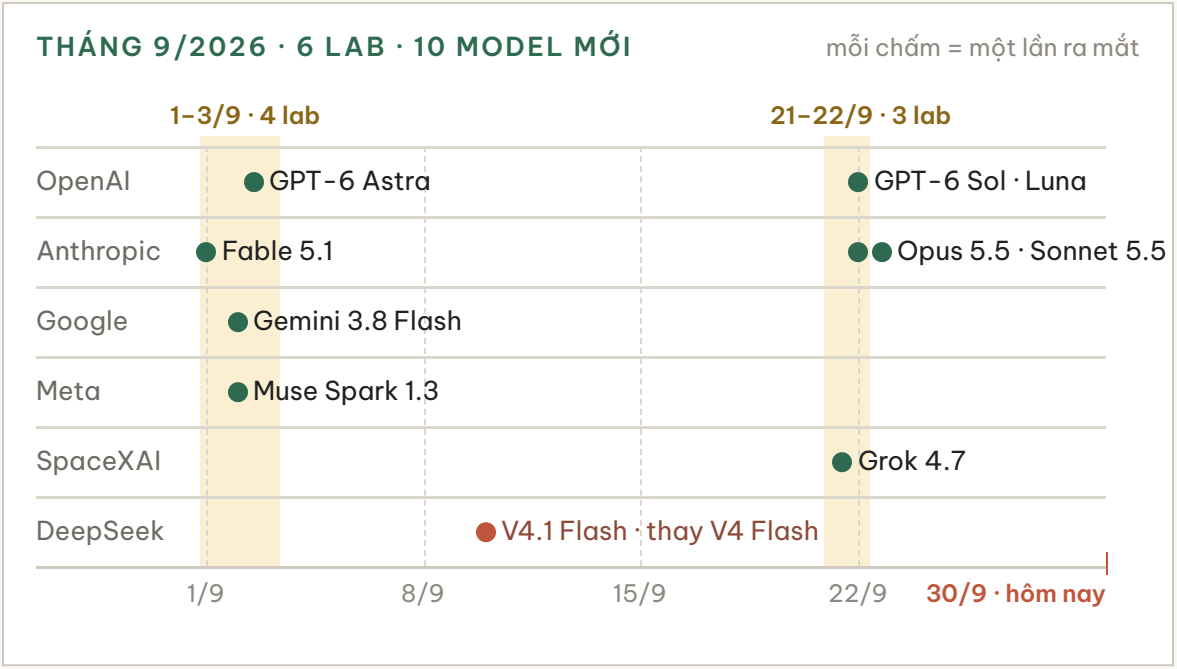
Generalist Product Builder

2026	FPT Long Châu · PM, Healthcare Product
2025	Thongtincuuho.org · Co-founder
2025	FPT Software AI Center · PM, AI Agent
2021–2025	Xantus · PM, On-chain Analytics & AI Agent
2016–2021	DYNO, Kalapa · PM, OCR & eKYC & Credit Scoring

LinkedIn | Facebook

Model mới ra hằng tuần — eval phải trả lời trong ngày

Mỗi model mới là một câu hỏi cho team: đổi sang thì sản phẩm của mình tốt lên hay tệ đi?



11 ngày

lại có một model frontier

Trung vị của 5 lab lớn ở Mỹ, 2026 (đến 28/4). Năm 2023: 37,5 ngày.

6 tuần

không đổi model cũng bị đổi

DeepSeek V4 Flash: ra 31/7, gỡ 10/9. Gọi tên cũ, bạn nhận V4.1 Flash.

Chưa dám đổi sang model mới ngay trong ngày nó ra? Nút thắt của team bạn là evaluation.

Nên bộ eval phải chạy lại được bất cứ lúc nào: input thật cố định, tiêu chí Pass/Fail rõ, phần lớn chấm tự động.

Agenda

01	**Đọc trace**	Lời agent nói là thứ user thấy — trace (= transcript) là thứ PM phải đọc, từng bước agent thực thi.
02	**Thiết kế dataset có coverage**	User Input Grid: LLM giúp sinh input, con người kiểm soát coverage và review lại.
03	**Cảm tính → tiêu chí đo được**	Thu hẹp scope, định nghĩa "đủ tốt", formalize thành rubric cả team chấm giống nhau.
04	**Chọn evaluator cho từng tiêu chí**	Cái gì kiểm bằng code, cái gì cần LLM judge, cái gì phải đến tay expert.
05	**Đưa expert vào loop**	Expert định nghĩa chuẩn vàng — dùng để calibrate judge và bắt failure mode mới.

BỐI CẢNH

Câu chuyện quen thuộc của mọi team làm AI

NGÀY DEMO

Demo chạy tốt

Khách hàng sớm khen, stakeholder đồng thuận. Bạn quyết định mở rộng.

→

VÀI TUẦN SAU

Bức tranh mờ dần

User đông hơn: người rất thích, người phàn nàn, người không quay lại. Đổi model — không biết còn ổn không.

→

HÔM NAY

Product Review ngày mai

Leadership hỏi bạn track chất lượng bằng cách nào. Usage metrics không trả lời được.

"AI của mình có đang làm việc tốt một cách ổn định không?"

— bạn không có câu trả lời. Đó là evaluation gap.

CÂU HỎI GIỮ SUỐT BÀI

Bạn vừa sửa prompt cho Support Triage Agent. Trên 50 ticket có nhãn, accuracy tăng từ 70% lên 78%, vượt ngưỡng ship 75%.

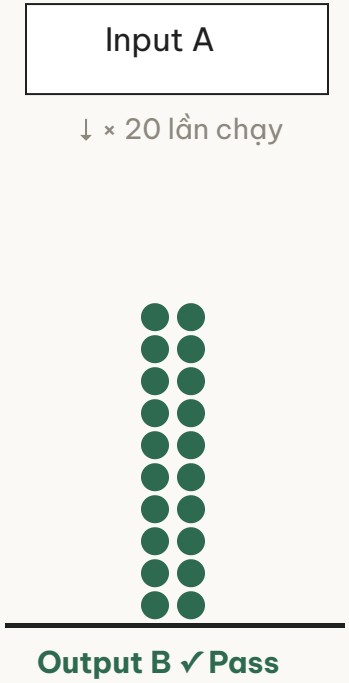
1. **Đủ để ship chưa?**

2. **Đủ để nói prompt mới tốt hơn chưa?**

Giơ tay cho từng câu. Ta trả lời ở Part 05.

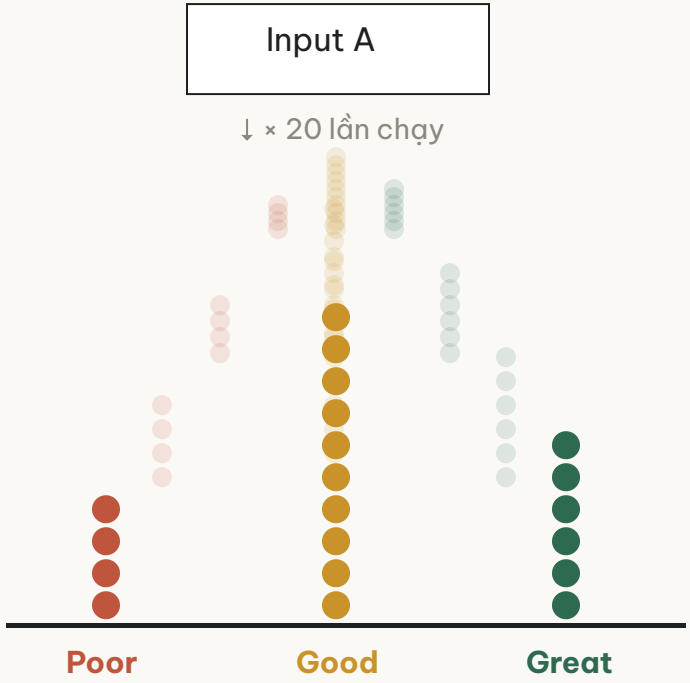
AI product: phân bố chất lượng

TRADITIONAL PRODUCT



Metric: Test Pass / Fail — 1 hoặc 0

AI PRODUCT



Metric: Agent Success Rate — một phân bố

AI Evals có hai lớp — product team làm lớp trên

PRODUCT TEAM · LÀM Ở ĐÂY

Application Evals

Chất lượng sản phẩm trong ngữ cảnh user thật

- Accuracy
- Helpfulness
- Tone / Style
- Safety
- Latency
- Format compliance
- User intent match
- Cost efficiency

MODEL PROVIDER

Model Evals

Benchmark chuẩn hoá cho năng lực nền

- Reasoning
- Coding
- Math
- Instruction following
- Safety

Model giỏi benchmark không đảm bảo sản phẩm đạt chất lượng — hai lớp đo hai thứ khác nhau.

CASE STUDY MỞ MÀN

Notion AI

Một team coi evals là IP của mình.

Notion AI dành 90% thời gian cho evals

Chỉ 10% là viết prompts.

100M+

users toàn cầu

Scale khổng lồ — không thể test bằng tay hay intuition.

60%

user Enterprise không nói tiếng Anh

100% engineer nói tiếng Anh — buộc phải dựa vào evals.

< 1 ngày

để ship model mới

Evals tự động cho biết ngay prompt nào regression khi model mới ra.

"Evals is our IP — everything we build, how we decide if those work well, lives inside Braintrust."

Sarah Sachs — AI lead, Notion

Nguồn: How to build world-class AI products — Sarah Sachs (AI lead @ Notion) & Carlos Esteban (Braintrust)

Evals là user research mới của PM

"You can think of this as your version of User Research." Sarah Sachs — AI lead, Notion

USER RESEARCH TRUYỀN THỐNG

User interviews
Hỏi user nghĩ gì — không phải user thật sự làm gì.

Surveys
Số lượng lớn nhưng context nghèo — khó biết tại sao user không hài lòng.

Chậm và tốn kém
Vài tuần để có insight — không kịp với tốc độ release AI.

EVALS AS USER RESEARCH — CÁCH NOTION LÀM

Thumbs-down data
Natural-language request thật từ user — không phải câu hỏi PM đặt ra trước.

Behavior thật, không phải claimed behavior
User muốn draft report, không chỉ research — tìm ra từ data, không từ interview.

Real-time, liên tục
PM ngồi trong eval tool mỗi ngày — insight cập nhật theo từng release.

PM không cần viết scoring function — nhưng phải là người đọc data và escalate findings. Việc này không ai làm thay được.

Notion quản lý evals như thế nào — ai làm gì

BƯỚC	PRODUCT MANAGER	ENGINEER	DATA SPECIALIST
1 · Decide Xác định vấn đề cụ thể	Đọc thumbs-down data và usage log để tìm điểm cần cải thiện.	Xác định cải tiến cần làm — ví dụ thêm một connector mới.	—
2 · Curate Tạo targeted dataset	Kiêm luôn vai data specialist ở team nhỏ.	—	Tạo dataset từ log thật. Bắt đầu với 10 samples, format đúng trước.
3 · Score Gắn scoring functions	—	Viết scoring function sau khi đã nhìn data lâu. Mix LLM-as-judge và heuristic.	Viết judge prompt riêng cho từng sample — tốn công nhưng sâu hơn nhiều.
4 · Inspect Review failures kỹ	Phát hiện pattern thật, rồi escalate findings lên roadmap.	Đưa failures vào một LLM để tóm tắt theme, ra báo cáo nhanh.	Human label failures. Quality quan trọng hơn quantity.

↺ Loop trở lại bước 1: chỉnh prompt theo failure theme, thêm rows dataset từ log mới.

Data lấy từ đâu, và dùng theo nguyên tắc nào

LẤY DATA TỪ ĐÂU

1 Dog-fooding — data tự nhiên

Team dùng sản phẩm của mình hàng ngày, nên mọi interaction đều là data tiềm năng. Không cần giả lập hay synthetic data từ đầu.

2 Thumbs-down — signal từ user

Chỉ lấy natural-language request, không lấy output cũ vì output lỗi thời rất nhanh. Request thật của user là tài sản dài hạn, dùng lại được mãi để eval model mới.

BA NGUYÊN TẮC KHI DÙNG DATA

Bắt đầu nhỏ: 10 samples, không phải 1000
Format đúng trước — dummy data sai cấu trúc là sai từ gốc.

Quality quan trọng hơn quantity
Đặc biệt khi fine-tuning — ít sample tốt hơn nhiều sample kém.

Data flywheel tích lũy theo thời gian
Mỗi release thêm log mới, dataset lớn dần, evals ngày càng chính xác hơn.

"The people closest to the data should be the ones creating the dataset."

Sarah Sachs — AI lead, Notion

Có hệ thống evals thì thay đổi những gì

× Trước

Dùng spreadsheet để label data
Human labeler quá tải, phải parse prompt bằng công thức bảng tính.

Model mới cần vài tuần để validate
Không biết prompt nào regression — phải test thủ công từng cái.

Ship dựa trên gut feeling
Chạy tốt trong demo — không biết có nhất quán với user thật không.

✓ Sau — với hệ thống evals

Model mới ship trong một ngày
Chạy toàn bộ evals, biết ngay regression, đổi model tự tin.

Tự tin build cho user đa ngôn ngữ
Multilingual evals đảm bảo chất lượng cho ngôn ngữ engineer không nói được.

Insight sản phẩm đến từ data
User muốn draft report — phát hiện từ thumbs-down, không từ interview.

"I don't think that we could exist without a tool like this today — it is critical to our iteration flow, and it actually is our IP."

Sarah Sachs — AI lead, Notion

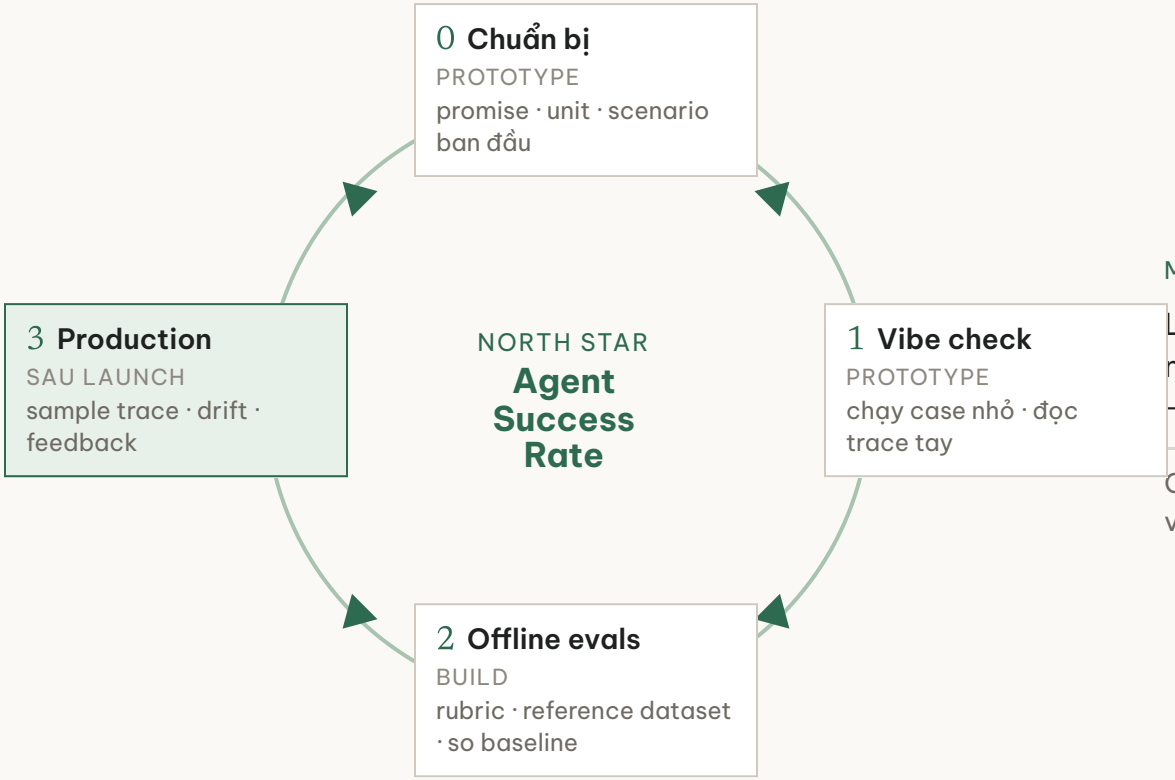
PART 01

AI Evals Lifecycle

Từ quan sát thủ công đến kiểm soát chất lượng liên tục.

Câu hỏi đo lường ⑥ Phép đo có còn đúng theo thời gian không?

Evals là một vòng lặp, không phải một lần kiểm thử



MŨI TÊN QUAY VỀ

Lỗi mới từ production → thành case mới trong dataset → rubric cập nhật → vòng sau chạy lại.

Các slide sau đi lần lượt từng chặng của vòng này.

Bước 1 — Vibe Check: chạy trước khi viết PRD

Mục tiêu là khám phá behavior, chưa phải chấm điểm chính thức.

1

Generate inputs

10–30 đầu vào bao phủ nhiều persona và use case.

happy path | edge case

2

Run prototype

Chạy từng đầu vào qua prototype, quan sát model vận hành thật.

input → agent → output

3

Label outputs

Gắn nhãn từng đầu ra, ghi lý do pass hoặc fail.

Ship | Chỉnh | Fail

4

Note patterns

Model làm được gì, không làm được gì, hành vi nào bất ngờ?

what works | surprises

FAIL LẶP LẠI → SEEDS YOUR PRD

Những gì model không làm được định hình chính xác những gì PRD cần đặc tả.

PASS TỐT NHẤT → SEEDS GOLDEN OUTPUTS

Output tốt nhất từ vibe check trở thành reference dataset — tách pool few-shot khỏi bộ eval.

Demo before memo — quan sát model trước, viết yêu cầu sau.

Output bước 1 — Golden Outputs

Reference dataset là câu thật cộng nhãn đúng kỳ vọng — dùng cho few-shot và cho evals — hai pool riêng, không chung một ví dụ.

#	CÂU NGƯỜI DÙNG	Ý ĐỊNH	CẢM XÚC	ƯU TIÊN
1	Mình nhận áo rồi mà chật quá, đổi size sao vậy?	Đổi hàng	Khó chịu	Trung bình
2	Shop giao nhầm màu cho mình rồi	Đổi hàng	Bực bội	Cao
3	App báo giao rồi mà mình chưa thấy đơn đâu	Tra cứu đơn	Tức giận	Cao
4	Mình muốn trả đôi giày hôm trước mà không còn mã đơn nữa	Trả hàng	Lo lắng	Trung bình
5	Cái đơn hôm bữa ấy, thôi giờ mình không lấy nữa được không	Cần làm rõ	Trung tính	Trung bình

PHẢI CÓ EDGE CASE

Câu mơ hồ · Thiếu thông tin · Nhiều ý trong một câu · Không nhớ mã đơn

"Áo không giống hình lắm, giờ đổi hay trả được ta"

"Mình đặt 2 đơn, 1 đơn muốn trả, 1 đơn muốn đổi size"

"Cái đơn hôm trước đó, check giúp mình với"

- ✓ Case mơ hồ giúp test khả năng hỏi lại
- ✓ Không ép model đoán khi chưa rõ
- ✓ Dataset tiếp tục lớn dần sau production

Bước 2 — Offline Evals: chấm dataset trước release

01 **Trigger thay đổi**
Prompt mới · model upgrade · tool integration

↓

02 **Chạy trên reference dataset**
Tự động, không cần manual review

↓

03 **So với baseline**
Version hiện tại là chuẩn so sánh

Deploy
Quality ok, approve

Fix & iterate
Regression — điều tra rồi chạy lại

Sau khi fix, quay lại bước 01 và chạy lại toàn bộ eval.

BENCHMARK CỐ ĐỊNH — MỌI RELEASE DÙNG CHUNG

Reference dataset

50 historical tickets, labels đã verify

Categorization accuracy eval

Sentiment precision eval

Escalation recall eval

Critical — không được phép regression

CẬP NHẬT DATASET KHI

Phát hiện failure mode mới từ production

Scope sản phẩm thay đổi

Evals quá dễ pass

"Nếu không thể ship model mới ngay trong ngày ra mắt — bottleneck là evaluation, không phải model."

Một kết quả eval nhìn như thế nào

DIMENSION	BASELINE	CANDIDATE	DELTA
Accuracy % output đúng rubric	70%	**78%**	+8%
Latency P50 (median) response time	1.2s	**1.8s**	+0.6s
Edge case pass rate % edge case xử lý đúng	44%	**61%**	+17%
Regression count Case từng pass, nay fail	—	**3**	Cần review
Quality gate Pass threshold ≥ 75%	70% — fail	78% — pass	Gate cleared

VERDICT

Ship

Đạt ngưỡng ≥75%, không regression critical. 3 regression trace cần đọc trước khi ship.

② Tốt hơn thật chưa? → Part 05

CẦN ĐỌC THÊM

3 regression traces — hiểu tại sao fail trước khi ship.

Số liệu cho phép ship. Nó không thay việc đọc trace.

Đừng chỉ nhìn con số tổng — **trung bình che khuyết điểm**: tổng score tăng 82%→85% vẫn có thể giấu một phân khúc tụt 70%→55%. Luôn đọc theo slice và mở regression traces trước khi ship. Và trên 50 ticket, mỗi ticket là 2 điểm %: **+8 chỉ là 4 ticket**. Chênh bao nhiêu mới là thật? Part 05 trả lời.

Cần bao nhiêu case để tin một khoảng chênh (95%)? **Chênh 10% → ~100 · 3% → ~1.000 · 1% → ~10.000** (OpenAI) — chênh nhỏ đi 3×, số mẫu cần tăng 10×.

Bước 3 — Sau launch, user tạo ra case bạn chưa nghĩ tới

SIGNAL MỚI TỪ PRODUCTION

Intent mới
User dùng AI theo cách team chưa nghĩ tới khi thiết kế

Ngôn ngữ và data format lạ
Tiếng địa phương, ký tự đặc biệt, input không chuẩn

Behavior ngoài use case ban đầu
User kéo AI về tình huống không nằm trong PRD

Tool và data source thay đổi
API third-party update, schema đổi, pipeline drift

User expectation thay đổi
Chuẩn "tốt" của user dịch chuyển theo thời gian dùng

Q1

Product còn đạt quality bar ngay lúc này không?

Agent Success Rate trong production có dưới ngưỡng?

REAL-TIME CHECK

Q2

Offline và online score có đang diverge?

Offline tốt mà online xấu — dataset chưa phản ánh thực tế.

DISTRIBUTION GAP

Q3

Failure mode nào mới xuất hiện?

Unknown unknowns — offline eval không thấy được cho tới khi có user thật.

NEW BLIND SPOTS

Q4

User feedback có mâu thuẫn với eval?

Thumbs-down tăng mà eval score ổn — rubric đang đo sai thứ.

SIGNAL CONFLICT

PRD cho sản phẩm AI cần thêm bốn mục — và một mục mới hẳn

TRADITIONAL PRD		AI-NATIVE PRD BỔ SUNG	TẠI SAO CẦN THÊM
Goals & success metrics DAU · Conversion · Retention	+	**Evaluation Rubric** Accuracy %, latency, pass/fail rubric	AI không có conversion rate — cần rubric riêng để đo chất lượng output.
User stories / use cases Luồng người dùng · Acceptance criteria	+	**Golden Outputs** Input → expected output cụ thể, test được ngay	"As a user, I want" không test được — cần output mẫu thật để eval.
Functional requirements Feature list · Business logic	+	**Prompt Logic & Tools** System instructions, API tools, failure rules	AI cần biết cách xử lý edge case — không chỉ "tính năng là gì".
Open questions Câu hỏi chưa rõ	+	**Dataset Strategy** Unknowns → dataset cần thu thập để eval	"Chưa rõ" phải thành "cần thu thập dataset này".
(không có tương đương)	MỚI	**Edge Case Handling** Khi model fail — thất bại thế nào là chấp nhận được?	AI fail theo nhiều cách — PRD phải định nghĩa "fail gracefully" nghĩa là gì.

"PRD định nghĩa quality bar – không chỉ danh sách tính năng."

Một AI PRD hoàn chỉnh — Support Ticket Triage v1.0

Sample AI PRD — v1.0 Status: Prototyping

1. **Problem & Business Value** — 4 giờ/ngày tag tay → agent phân loại real time
2. **Prompt Logic & Dataset** — system instruction + 50 golden tickets đính kèm
3. **Tool Specification** — 4 API agent được phép gọi, và để làm gì
4. **Evaluation Criteria** — target đo được cho từng metric
5. **Edge Cases Handling** — fallback khi model không chắc chắn
6. **Prototype & Early Findings** — những gì chạy thử dạy cho spec
7. **Technical Constraints** — PII scrubbing, cost < $0.01/ticket

▪ Mục 4–6: phần không tồn tại trong PRD truyền thống

Quality bar viết thành số, không phải tính từ

Accuracy > 92% | Sentiment > 85% | Latency < 2s | Bịa ID: 0 lỗi / ≥ 300 case

Fail gracefully được thiết kế trước

Confidence < 0.7 → không auto-tag, chuyển Needs Human Review · audit tay 5% ticket mỗi tuần · nút "Tag này đúng không?" cho support.

PRD viết SAU khi prototype

Findings thật quay lại sửa spec: input quá ngắn gây hallucinate → thêm chế độ hỏi lại · 20% ticket có 2 issue → cần multi-label · sarcasm → thêm 5 ví dụ vào golden dataset.

Định nghĩa tiêu chí chất lượng

Thu hẹp cho đến khi viết được rubric Pass/Fail mà hai người chấm độc lập cho cùng kết quả.

QUÁ RỘNG — KHÔNG ĐO ĐƯỢC

“Cải thiện trải nghiệm hỗ trợ khách hàng bằng AI”

↓ **Trục 1 · Tách theo task**

Phân loại ticket · soạn trả lời · quyết định escalate · cập nhật CRM

↓ **Trục 2 · Tách theo điều kiện input**

Ticket rõ ràng · ticket mơ hồ — kỳ vọng khác: hỏi lại thay vì đoán

↓ **Trục 3 · Tách theo mức tự trị**

UNIT OF AI WORK

Phân loại 1 ticket → đúng 1 trong 3 nhóm, kèm sentiment, dưới 2 giây

✓ Viết được rubric Pass/Fail — sẵn sàng đo

PART 02

Chạy agent, đọc trace, chuẩn hoá chất lượng

Đừng bắt đầu bằng automation — đọc trace trước.

Câu hỏi đo lường ① Ta đang đo cái gì?

Muốn tìm lỗi, trước hết phải có trace

Một trace đủ để người review dựng lại chuyện gì đã xảy ra — và quyết định nó tốt hay không.

TRACE THÔ · TRỢ LÝ CHO THUÊ CĂN HỘ

… = phần lược bớt

```
"tools": [ { "name": "getCommunitiesAvailability",                ← Tool được dùng
             "description": "… apartments …" } ],
"messages": [
  { "role": "system", "content": "… leasing team member …" },     ← System prompt
  { "role": "user",   "content": "Total move-in                   ← Input của user
                                  package for 2be 2ba …" },
  { "role": "tool",   "content": "{apartments:[…]}" },           ← Kết quả tool
  { "role": "assistant", "content": "… rent ranges from           ← Output
                                  $1,995 to $2,025. …" },
  { "role": "user",   "content": "This is out of my budget. …" },
  { "role": "assistant", "content": "You're welcome! If your     ← Lỗi: chào tạm biệt
                                  situation changes …" }
]
```

Khách vừa nói quá ngân sách, agent lại chào tạm biệt. Chỉ đọc câu cuối thì nghe rất lịch sự — phải đọc cả trace mới thấy lỗi.

CHƯA CÓ TRACE?

1 **Nhờ coding agent instrument app**

```
Log mỗi phiên thành
một trace: input,
system prompt, tool
call + kết quả,
context, output.
```

2 **Chờ người dùng thật hoạt động**

Lưu vào database, thư mục local, hoặc công cụ tracing (dải bên dưới).

3 **Chưa launch → trace tổng hợp**

Dùng LLM giả lập câu hỏi của user. Không thay được data thật — nhưng hơn là không có gì.

CÔNG CỤ TRACING: Langfuse | Braintrust | Arize Phoenix | LangSmith | W&B Weave

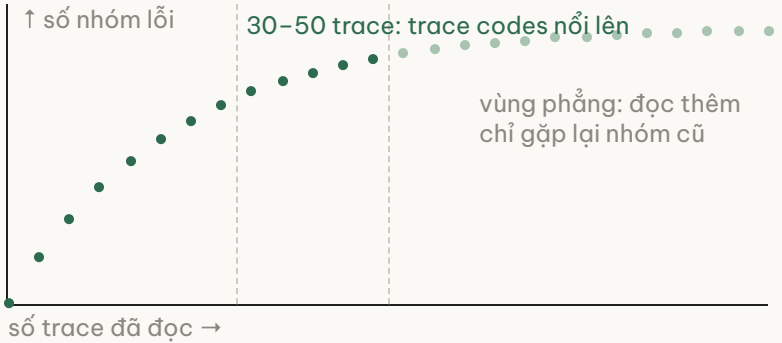
Nguồn: Hamel Husain & Shreya Shankar — ví dụ trace và prompt instrument

Đọc trace nào, và đọc bao nhiêu thì dừng

TRỘN NĂM NGUỒN

Ngẫu nhiên ≥30% MỖI ĐỢT	Nguồn duy nhất không bị lọc qua thứ hệ thống đã biết
Đại diện mỗi cụm CỤM	Mỗi cụm câu hỏi giống nhau lấy 2–3 case
Outlier OUTLIER	p99 latency, gọi nhiều tool nhất, trả lời dài nhất
Đã bị flag ĐÃ FLAG	Guardrail chặn, check tự động báo đỏ
Feedback xấu FEEDBACK	Thumbs-down, hỏi lại ngay, bỏ phiên giữa chừng

ĐỌC ĐẾN KHI BÃO HOÀ



Dừng khi 10–20 trace liên tiếp, đủ loại khác nhau, không sinh nhóm mới — thường ~100 trace, 2–3 giờ.

Một người đọc liền mạch — chia 5 người thì nhóm lỗi của mỗi người không khớp.

> Chỉ đọc case đã bị flag thì bạn chỉ gặp lại đúng những lỗi hệ thống vốn đã bắt được.

4 quy tắc khi ghi chú lúc đọc trace

1 Viết cụ thể — người khác đọc là hiểu

×"Trả lời tệ" · ✓"Nói bài có dạy sâu về retrieval nhưng slide bài không có phần này"

2 Ghi điều user gặp, chưa đoán nguyên nhân

Ghi triệu chứng user thấy. Đoán "vì sao" để sau, khi đã gom nhóm.

3 Mỗi trace chỉ ghi lỗi gốc — lỗi xảy ra đầu tiên

Các lỗi sau thường là hệ quả của lỗi đầu. Đếm cả thì nhóm lỗi lớn nhất sẽ bị sai. Ví dụ SC-042: ghi lỗi ở bước truy xuất, không ghi ở câu trả lời.

4 Ghi tự do trước, chưa dùng checklist

Chưa mở checklist nào — checklist chỉ cho bạn thấy những gì nó đã liệt kê.

VÍ DỤ GHI CHÚ ĐÚNG CÁCH

```
SC-042 — nói bài có dạy sâu về retrieval, slide bài không có phần này
TKT-00421 — câu trả lời nhắc mã TKT-00422, không có trong yêu cầu
TKT-00507 — hứa hoàn tiền trước khi tra cứu đơn
```

Chuẩn hoá notes thành trace codes

Notes tự do sau vòng discovery

Mỗi người nói một kiểu.

“con bot này xử lý sai ý khách”

“nó hơi bị lạc hướng”

“nó không hiểu khách muốn đổi hàng”

“agent trả lời mà không tra cứu đơn”

“agent hứa hoàn tiền quá sớm”

“agent không chuyển human khi phức tạp”

Khó đo Khó so sánh Khó sửa

formalize →

Trace codes — ngôn ngữ chung của team

Cùng một vấn đề, cùng một tên.

wrong_intent
Agent hiểu sai nhu cầu chính của user

missing_lookup
Không tra cứu dữ liệu trước khi trả lời

premature_commit
Hứa kết quả khi chưa đủ điều kiện

missing_escalation
Không chuyển human khi vượt thẩm quyền

no_next_step
Đúng policy nhưng thiếu hành động tiếp theo

Đo được So sánh được Sửa được

Một code dùng được khi **mọi trace trong nó đóng lại bằng cùng một lần sửa** — gom theo chỗ phải sửa, không theo triệu chứng. Test nhanh: đọc tên xong, viết được ngay câu hỏi có/không chưa?

Taxonomy phải tuỳ theo use case

Không có một taxonomy dùng chung — xây từ trace analysis của chính mình.

AI Support đơn hàng

Xoay quanh: intent · order · policy · authority

- wrong_intent
- missing_order_lookup
- policy_misapplied
- premature_refund_commit
- missing_escalation
- unclear_next_step

AI Learning Assistant

Xoay quanh: retrieval · grounding · boundary

- wrong_question_intent
- poor_retrieval
- unsupported_claim
- external_boundary_violation
- poor_followup_quality

AI Travel Planner

Xoay quanh: feasibility · data · authority

- missing_constraint
- stale_live_data
- infeasible_schedule
- budget_violation
- booking_overreach
- missing_recovery

Đừng copy taxonomy từ bài khác — dùng trace analysis để xem use case của mình thật sự fail ở tầng nào.

> Agent có quyền ghi: bộ test phải có case **thiếu tham số**. “Hủy đơn gần nhất” mà không có mã đơn → agent tự đoán và gọi `cancel_order` với đơn của người khác.

Error discovery: người nhận ra, agent sắp xếp

Người luôn đi trước — để không bị neo theo gợi ý của máy.

	1 · 10 TRACE ĐẦU	2 · GỢI Ý	3 · DUYỆT	4 · LẶP	5 · GOM & ĐẾM
Người	Tự đọc, ghi lỗi — **chưa xem gợi ý máy**		Chấp nhận, bác, sửa — theo dõi tỉ lệ chấp nhận	Đến ~100 trace hoặc bão hoà	Chốt trace codes và thứ tự ưu tiên
Agent		Đề xuất ghi chú cho trace tiếp theo		Học từ chỗ người bác	Gom ghi chú thành failure mode, đếm

Máy tìm lại được **74–87%** số lỗi chuyên gia gán.

Nhưng sót **toàn bộ** lỗi cần ngữ cảnh ngoài trace: mục tiêu kinh doanh, kênh, quy định.

Husain & Shankar 2026 · Saha & Husain 2026, 100 trace production

Đếm trước, sửa sau: ưu tiên = tần suất × mức nặng

AI Support đơn hàng · 100 trace đã đọc · số minh hoạ

TRACE CODE	SỐ TRACE	MỨC NẶNG	TẦN SUẤT × MỨC NẶNG	THỨ TỰ
`missing_order_lookup`	18	Trung bình ×2	36	1
`premature_refund_commit`	5	**Cao ×5 — hứa hoàn sai = mất tiền**	25	2
`wrong_intent`	12	Trung bình ×2	24	3
`missing_escalation`	4	**Cao ×5 — khách phức tạp bị bỏ rơi**	20	4
`unclear_next_step`	9	Thấp ×1	9	5

5 lần hứa hoàn tiền sai đáng lo hơn 12 lần hiểu sai ý. **Trọng số mức nặng là quyết định sản phẩm — ghi ra trước, như ngưỡng ship.**

PART 03

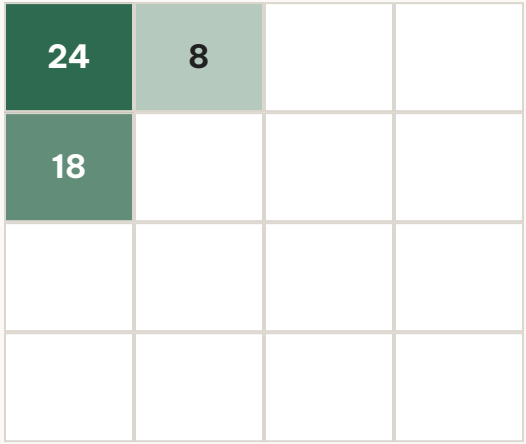
Thiết kế coverage & candidate scenarios

Chọn biến làm agent phải hành xử khác đi, trước khi viết test prompt.

Vấn đề của “hãy tạo 50 test prompts”

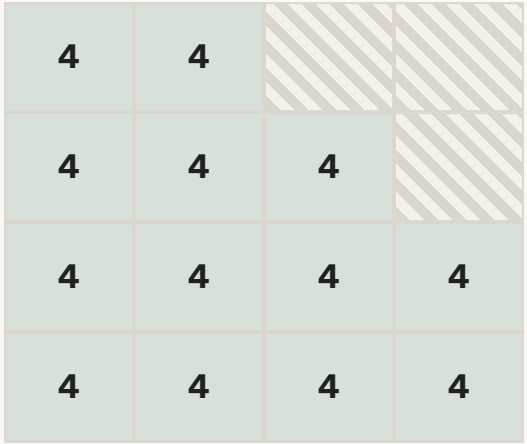
Cùng 50 prompt: để LLM tự sinh vs thiết kế theo grid Persona × Intent

BƯỚC 1 · LLM TỰ SINH



3/16 ô có test · 47/50 dồn vào happy path

BƯỚC 2 · THEO GRID PERSONA × INTENT



13/13 ô hợp lý có test · ô gạch = phi lý, bỏ

LLM sinh **nhiều**, con người quyết **phủ ở đâu**.

Thuộc tính nào là dimension? Nhìn vào cột hành vi đúng

Cùng một phép thử cho hai thuộc tính của input: đổi giá trị → hành vi đúng có đổi không?

DIMENSION? Cách diễn đạt — ✕ Không – bỏ

INPUT – 3 GIÁ TRỊ CỦA THUỘC TÍNH	HÀNH VI ĐÚNG
GIÁ TRỊ: LỊCH SỰ "Tôi muốn hoàn tiền đơn #4521."	→
GIÁ TRỊ: CỘC LỐC "Refund #4521!"	→ Vẫn chỉ MỘT hành vi: kiểm tra chính sách → tạo yêu cầu hoàn
GIÁ TRỊ: CÂU HỎI "Đơn #4521 hoàn tiền được không?"	→

3 giá trị → 1 hành vi. Chỉ là paraphrase – để LLM tự sinh, không tốn một trục grid.

DIMENSION? Độ đủ thông tin — ✓ Là dimension

INPUT – 3 GIÁ TRỊ CỦA THUỘC TÍNH	HÀNH VI ĐÚNG
GIÁ TRỊ: ĐỦ THÔNG TIN "Hoàn tiền đơn #4521, giao trễ 5 ngày."	→ Xử lý luôn
GIÁ TRỊ: THIẾU MÃ ĐƠN "Hoàn tiền đơn của tôi."	→ Hỏi đúng mã đơn
GIÁ TRỊ: MƠ HỒ "Đơn vừa rồi tệ quá!"	→ Làm rõ ý định trước

3 giá trị → 3 hành vi khác nhau. Thành một trục của grid – mỗi giá trị phải có mặt trong dataset.

Đọc từ trên xuống: **Thuộc tính** (nhãn Dimension?) → **3 giá trị** (tag trên mỗi input) → **hành vi đúng**. Hành vi đổi theo giá trị → thuộc tính đó là dimension.

Chọn dimension — biến nào tạo ra sự đa dạng có ý nghĩa?

Một phép thử duy nhất: đổi giá trị của biến → hành vi đúng của agent có phải đổi không?

① BIẾN ỨNG VIÊN

- ICP
- Persona
- User Intent
- Context Richness
- Ambiguity Level
- Giọng lịch sự / cộc
- Độ dài câu
- Lỗi chính tả

5 dimension phổ biến (Module 04) · 3 biến hay bị nhầm

② QUA PHÉP THỬ

Đổi giá trị → hành vi đúng đổi?

Có → dimension, thành một hàng của grid

Không → paraphrase, để LLM tự sinh biến thể

③ GRID AI TUTOR · MỖI TEST CASE = CHỌN 1 Ô MỖI HÀNG

Đáp án trong tài liệu?	Có trực tiếp	**Phải tổng hợp**	Ngoài phạm vi
Người học	**Học viên mới**	Đã học qua	Ôn thi
Intent	**Hỏi khái niệm**	Tra cứu bài	Xin lộ trình
Độ rõ câu hỏi	Rõ, đủ ngữ cảnh	**Thiếu ngữ cảnh**	Nhiều ý

CÂU HỎI SINH RA

"Eval với test khác nhau chỗ nào nhỉ? Bài nào cũng nhắc mà em chưa hiểu."

HÀNH VI ĐÚNG

Hỏi lại bài đang học · tổng hợp nhiều bài · cite từng nguồn · giải thích từ nền

Đổi một ô — ví dụ "Ngoài phạm vi" — hành vi đúng đổi hẳn: từ chối, nói rõ giới hạn.

Quy trình tạo User Input Grid

Thiết kế coverage trước khi viết test prompt. Đã có trace thật? Lấy input từ trace trước — grid để tìm ô còn trống.

BƯỚC 1

Chọn dimensions

Yếu tố nào làm expected behavior thay đổi?

Loại câu hỏi của user

Độ phủ tài liệu khóa học

Có cần research ngoài không

Ngôn ngữ và expertise

BƯỚC 2

Định nghĩa values

Đổi value thì câu trả lời đúng cũng phải đổi theo.

Có slide trả lời trực tiếp

Cần tổng hợp nhiều module

Course chỉ cover một phần

Course chưa đề cập

BƯỚC 3

Tạo combinations

Giữ lại combination khi:

Có khả năng xuất hiện thật

Làm agent phải đổi chiến lược

Failure cost cao

Team chưa chắc quality boundary

BƯỚC 4

Thêm constraint đời thật

Làm scenario bớt "sạch", giống user thật.

Hỏi mà không rõ bài nào

Slide và transcript dùng thuật ngữ khác

Source cũ vẫn đứng đầu retrieval

Answer đúng nhưng follow-up lặp lại

BƯỚC 5

Viết thành user inputs

LLM chỉ paraphrase — cùng scenario, nhiều cách diễn đạt.

ngắn / dài

formal / informal

tiếng Việt / tiếng Anh

beginner / advanced

× Không giao cho LLM tự chọn test coverage — human kiểm soát coverage, LLM chỉ diễn đạt lại.

PHỄU TỔ HỢP $4 \times 4 \times 4 = 64$ tổ hợp → loại phi lý → 15–20 scenario → ×2–3 diễn đạt → 40–50 test inputs

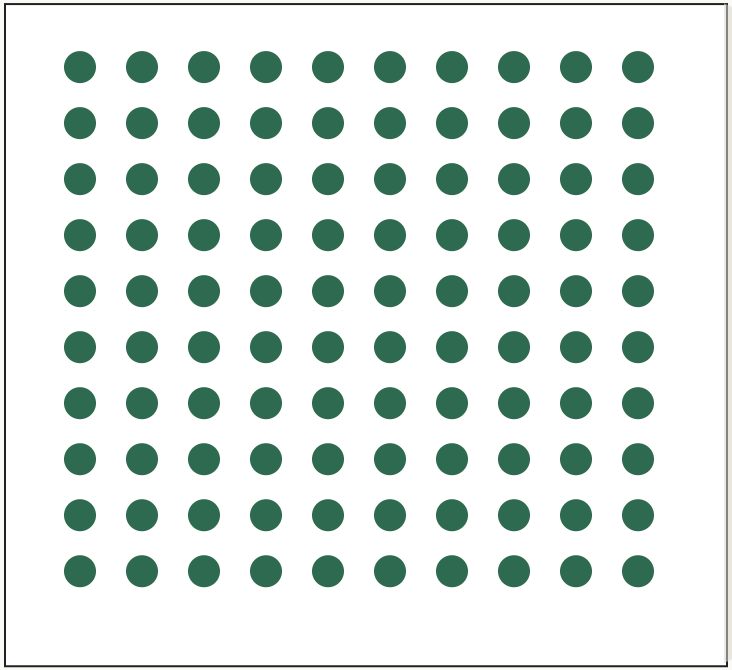
PART 04

Automated Evaluators

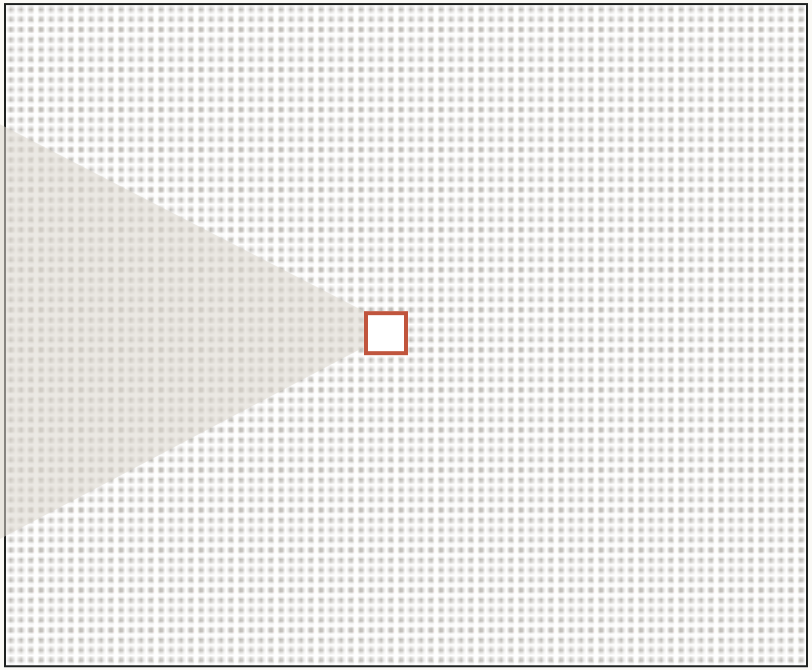
Khi nào nên automate · Code-based vs LLM-judge · Common mistakes

Manual review không thể scale

Automation mua lại: ship nhanh hơn · bắt drift trước khi user thấy · đo được mới cải thiện được.



100 traces / tuần — đọc tay được, hiểu pattern



100.000 traces — 100 trace bạn đọc được chỉ là **ô đỏ** này

Không phải vấn đề nào cũng cần eval

Ba câu hỏi trước khi viết eval cho một trace category — chỉ **generalization gap** mới đáng tự động hoá.

CÂU HỎI 1 · SPECIFICATION CHECK

Prompt có được giao xử lý tình huống này không?

KHÔNG →

Specification gap — thiếu đặc tả

Sửa system prompt — chưa cần viết eval.

Email drafter không tạo subject line vì prompt chưa hề yêu cầu.

↓ Có, prompt đã rõ

CÂU HỎI 2 · SYSTEM CHECK

Agent có bao giờ làm được việc này không?

KHÔNG BAO GIỜ →

Architectural issue — thiếu năng lực hệ thống

Engineering fix — thiếu tool, integration, hoặc model capability.

Calendar API chưa connect — prompt rõ đến đâu cũng không làm được.

↓ Có lúc làm được

CÂU HỎI 3 · GENERALIZATION CHECK

Lúc được lúc không — dù prompt đã rõ?

CÓ →

Generalization gap — thiếu tính nhất quán

Viết automated eval ✓ — đây là ứng viên phù hợp.

Email drafter đôi khi cá nhân hoá đúng, nhưng chỉ ở case rất rõ ràng.

Bản đồ: đo ở tầng nào trong hệ thống

Mỗi tầng hỏng theo kiểu riêng — và có cách đo rẻ nhất riêng.

1 Định tuyến	2 Truy xuất	3 Tool / hành động	4 Tổng hợp	5 Guardrail	6 Output
HỎNG hiểu sai yêu cầu; đoán thay vì hỏi lại khi thiếu thông tin ĐO BẰNG code so nhãn intent; case thiếu tham số: có hỏi lại không?	HỎNG lấy sai, thiếu, cũ; đúng nhưng xếp quá sâu ĐO BẰNG code: tài liệu đúng có trong top-k không (cần gold doc) RAGAS	HỎNG sai tool, sai tham số, sai thứ tự ĐO BẰNG code: so trạng thái cuối + thứ tự gọi	HỎNG bịa ngoài nguồn, trích sai ĐO BẰNG tách từng ý, đối chiếu với nguồn RAGAS	HỎNG chặn nhầm hoặc để lọt ĐO BẰNG bộ case tấn công + case phải từ chối	HỎNG sai format, sai nhãn; giọng, độ hữu ích ĐO BẰNG code (schema, nhãn) → judge (giọng) RAGAS

Cả phiên (session) — **Hỏng:** quên ràng buộc từ lượt đầu, đi sai nhánh — **Đo bằng:** chấm theo phiên, chạy lại k lần

> Đọc trace tìm lỗi gốc → đo đúng tầng đó. RAG chỉ là một tầng trong bản đồ này.

Tiêu chí nào giao được cho máy? Hỏi một câu duy nhất

> "Tiêu chí giao được cho máy khi và chỉ khi nó có một **referent kiểm chứng được** – một thứ bên ngoài phán đoán của model mà kết luận có thể quy về."

REFERENT RÕ · QUY TẮC, SCHEMA, TÀI LIỆU GỐC

Máy chấm – người kiểm mẫu

Máy ra quyết định; người kiểm ~10% ngẫu nhiên để canh drift.

Code-based – mặc định ưu tiên

- đúng format, đủ trường bắt buộc
- gọi đúng tool
- không lộ thông tin cấm
- đúng schema

Nhanh · Rẻ · Ổn định · Gắn critical path · Dễ debug

REFERENT MỘT PHẦN · SẮC THÁI, NGỮ CẢNH

Máy sàng lọc – người thẩm định

Máy xếp hạng nghi ngờ và nêu bằng chứng; người quyết mọi case bị cờ.

LLM-as-Judge – khi code không đủ

- hữu ích với đúng người dùng?
- giải thích hợp lý, nhất quán?
- diễn đạt đủ thuyết phục?
- đúng brand voice?

phải calibrate trước khi tin

KHÔNG CÓ REFERENT · CHỈ CÓ PHÁN ĐOÁN NGƯỜI

Người quyết – máy nêu bằng chứng

Máy không được xuất điểm số – chỉ xuất dữ kiện trung gian cho người xem.

Human / Expert review

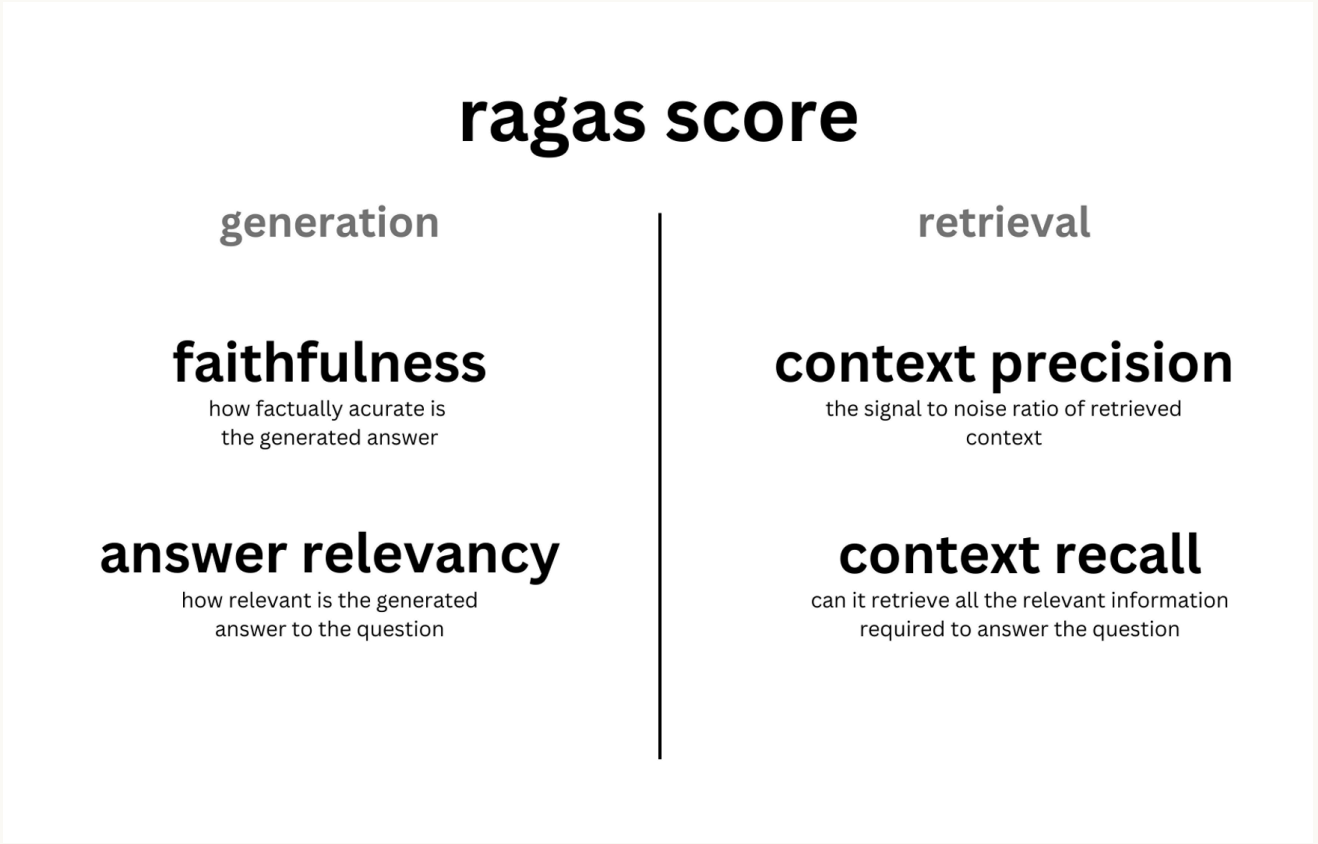
- chuẩn chuyên môn
- tiêu chí team chưa thống nhất được

không bao giờ giao máy chấm

Rule rõ → code. Cần sắc thái → LLM judge. Chỉ người phán được → người. Code trước, lùi sang phải khi buộc phải lùi.

Zheng et al. 2023 · Krumdick et al. 2025

RAGAS


ragas score
generation
retrieval
faithfulness
how factually acurate is
the generated answer
answer relevancy
how relevant is the generated
answer to the question
context precision
the signal to noise ratio of retrieved
context
context recall
can it retrieve all the relevant information
required to answer the question

PART 05

Code-based Evaluation

Deterministic checks chạy tự động trên mọi prompt change — nền tảng không thể thương lượng của eval suite.

Câu hỏi đo lường ⑤ Con số chính xác đến đâu?

PYTHON · DETERMINISTIC · KHÔNG GỌI LLM

Ba phần của một code-based eval

```
① INPUT
def check(output_str, user_input):
② LOGIC
  if "Subject:" not in output_str:
③ RESULT + REASON
    return False, "Missing subject line"
  return True, None
```

① Input — lấy đúng phần cần kiểm tra

Output string, JSON, tool calls — hoặc kết hợp cả user input khi cần kiểm tra theo ngữ cảnh.

② Logic — rule đủ rõ để máy quyết định

Condition, pattern match, schema validation — nếu viết được "phải có X" hay "đúng format Z" thì dùng code.

③ Result — pass/fail kèm reason string

Reason string không phải "có thì tốt" — khi fail ở trace 38/50, team phải biết ngay tại sao, không cần mở lại trace để đoán.

Category checking — Support Triage Agent

Agent phải chọn đúng 1 trong 3 labels: Technical · Billing · Feature Request

```
VALID = {"Technical", "Billing", "Feature Request"}

def check_category_label(output: str) → dict:
  found = [c for c in VALID if c == json.loads(output)
  ["category"]]

  if len(found) == 1:
    return {"pass": True,
      "reason": f"Single valid: {found[0]}"}
  elif len(found) == 0:
    return {"pass": False,
      "reason": "No valid category found"}
  else:
    return {"pass": False,
      "reason": f"Multiple labels: {found}"}
```

Danh sách labels được phép

Chỉ 3 giá trị exact string. “billing issue” hay “tech problem” không được tính.

Đếm bao nhiêu label xuất hiện

Quét output, gom tất cả label tìm thấy — rỗng, 1 phần tử, hoặc nhiều hơn.

Ba trường hợp, một kết quả rõ ràng

pass	Đúng 1 → ghi rõ label nào
fail	Không có → “No valid category found”
fail	Nhiều hơn 1 → liệt kê cụ thể để debug

Bốn nơi nên bắt đầu với code-based evals

Output có đúng cấu trúc không?	STRUCTURE / FORMAT	Đủ field, đúng format, đúng schema, trong giới hạn cho phép. Quan trọng khi output đi tiếp vào UI, database hoặc downstream system.
Output có đủ phần bắt buộc không?	PRESENCE / COVERAGE	Có nhắc đúng product name, ticket ID, required policy, source, next step? Bắt những output nghe ổn nhưng chưa đủ để user hành động.
Agent có đi đúng quy trình không?	TOOL-CALL / SEQUENCING	Gọi đúng tool, đúng thứ tự, đúng parameter. Nhiều lỗi không nằm ở câu trả lời cuối mà nằm ở cách agent thực thi workflow.
Chỉ số vận hành có trong ngưỡng không?	THRESHOLD CHECKS	Latency, cost, token count, confidence score có vượt ngưỡng không? Trả lời đúng nhưng chậm hơn hoặc đắt hơn nhiều vẫn là regression.

Output phải dùng được · nội dung phải đủ ý · quy trình phải đúng · chi phí phải trong ngưỡng.

Case study — ba kiểm tra tự động cho Support Triage Agent

KIỂM TRA 1

Câu trả lời có dùng đúng nhãn chuẩn không?

Phải chọn đúng một trong ba nhãn. Diễn đạt lại sẽ làm hỏng hệ thống phân loại phía sau.

✓ Đúng
Danh mục: Billing — khách khiếu nại hoá đơn tháng 3.

× Không tìm thấy nhãn hợp lệ
“Đây có vẻ là vấn đề liên quan đến thanh toán.”

KIỂM TRA 2

Agent có bịa thông tin không có trong yêu cầu không?

So sánh mã ticket trong câu trả lời với mã trong yêu cầu ban đầu.

✓ Đúng
Yêu cầu có TKT-00421 · câu trả lời chỉ nhắc TKT-00421.

× Bịa ra mã TKT-00422
Yêu cầu có TKT-00421 · câu trả lời nhắc TKT-00422.

KIỂM TRA 3

Thời gian phản hồi có còn trong ngưỡng không?

Câu trả lời đúng nhưng chậm hơn nhiều vẫn là lỗi cần bắt.

✓ Đúng
Thời gian 1.1 giây · ngưỡng tối đa 2.0 giây.

× Vượt ngưỡng 2.0 giây
Thời gian 2.4 giây.

Sau khi cập nhật prompt: cả ba pass → mới xem xét release.

Đây là mức sàn tối thiểu — câu hỏi tinh tế hơn vẫn cần người đọc và đánh giá.

PART 06

LLM Judge Calibration

Hiệu chỉnh bộ chấm để biết nó có đúng, không chỉ biết nó nghĩ gì.

Câu hỏi đo lường ④ Dụng cụ đo có đúng không?

LLM Judge — model thứ hai chấm output của model thứ nhất

Một prompt judge tốt trông như thế này — mỗi phần là một nguyên tắc.

PROMPT CỦA JUDGE

1. **MỘT TIÊU CHÍ DUY NHẤT**
   `Chỉ chấm: agent có ghi nhận cảm xúc khách TRƯỚC khi xử lý không?`
2. **VERDICT NHỊ PHÂN**
   `Trả lời Pass hoặc Fail. Không cho điểm 1-10.`
3. **VÍ DỤ SÁT RANH GIỚI**
   `Pass sát ranh: "Mình hiểu bạn đang bực…" · Fail sát ranh: "Đơn của bạn bị trễ."`
4. **CÙNG NGỮ CẢNH NGƯỜI CHẤM THẤY**
   `Trạng thái đơn: trễ 5 ngày, khách đã nhắn 2 lần.`
5. **CÂU TRẢ LỜI MẪU (NẾU CÓ)**
   `Có mẫu thì đưa vào — sai giảm từ 14/20 còn 3/20.`

→

OUTPUT CỦA JUDGE

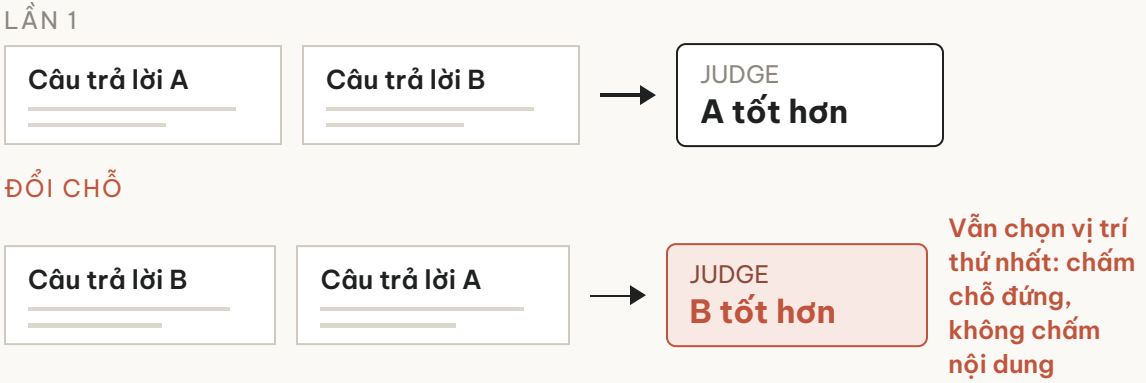
6. `Reasoning: agent nhắc sự việc nhưng đi thẳng vào quy trình, không ghi nhận cảm xúc…`
   `Verdict: Fail`

Reasoning viết trước — verdict là token cuối.

7. Judge khác họ model với agent — chấm bài "người nhà" dễ cho qua lỗi hơn 50%.

Verdict của judge là một **ý kiến**, không phải sự thật. Đo độ tin của ý kiến đó chính là **calibration**.

Giám khảo nào cũng thiên lệch — thiết kế cách chấm để nó tự triệt tiêu



66/80

câu Vicuna-13B "thắng" ChatGPT chỉ nhờ đổi thứ tự · Wang 2023

65%

số lần GPT-4 giữ nguyên phán quyết khi đổi chỗ · Zheng 2023

14 → 3

lỗi chấm trên 20 bài toán khi đưa đáp án tham chiếu · Zheng 2023

THIÊN LỆCH → CÁCH TRIỆT TIÊU

Vị trí
→ chấm cả hai thứ tự, chỉ tính khi hai lần khớp nhau

Độ dài
→ so hai câu trả lời dài tương đương, kiểm soát độ dài

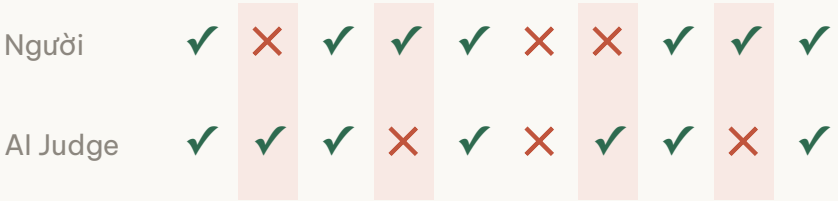
Người nhà
→ judge khác họ model với agent

Thiếu kiến thức
→ đưa đáp án tham chiếu vào prompt

Đừng hỏi "judge có thiên lệch không?" — có. Hỏi: **cách chấm đã làm thiên lệch triệt tiêu chưa?** Đổi chỗ, cân độ dài, khác họ model, có tham chiếu.

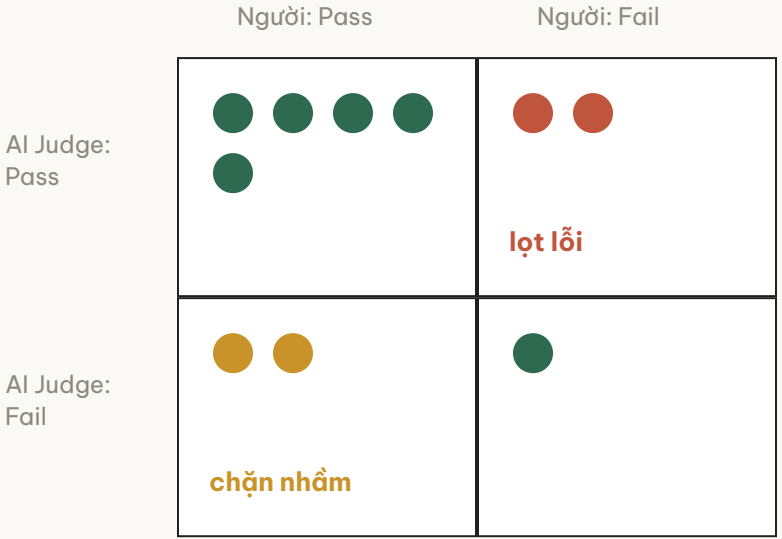
Cùng pass rate — chưa chắc AI Judge nghĩ giống bạn

Calibration = so phán quyết của AI Judge với nhãn của người trên cùng một bộ trace, từng dòng một.



Người 7/10 Pass · AI Judge 7/10 Pass — y hệt 70%

...nhưng lệch nhau 4/10 dòng



TPR 5/7 = 71% — nhận đúng output tốt

TNR 1/3 = 33% — bắt được output lỗi

Trong deck này: TPR = nhận đúng output tốt · TNR = bắt được output lỗi. Tài liệu khác có thể đảo quy ước.

Sáu bước calibration

Ba bước chuẩn bị → một vòng lặp đến khi đạt ngưỡng → kiểm tra đúng một lần.

1 — Label 50–100 trace
Expert chấm Pass/Fail, chia dev / test. **Đếm số case lỗi**, không đếm tổng.

→

2 — Chạy judge trên dev
Ghi verdict + reasoning cạnh nhãn người.

→

3 — Confusion matrix
Tính TPR và TNR.

→

↻ LẶP ĐẾN KHI ĐẠT NGƯỠNG

4 — Đọc TỪNG bất đồng
Hiểu sai tiêu chí · thiếu ví dụ · judge dễ dãi

5 — Sửa prompt — một thứ mỗi lần
Đòn mạnh nhất: thêm 3–4 near-miss

→

6 — Validate — MỘT lần
Chạy tập test giữ riêng. Chênh dev ≤ 5 điểm → dùng được.



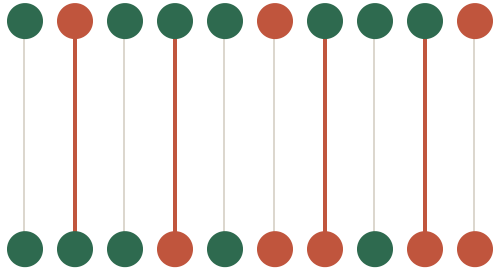
CASE EMPATHY JUDGE · TPR / TNR

TNR thấp → siết định nghĩa Fail + thêm near-miss · **TPR thấp** → thêm ví dụ Pass sát ranh · **<2–3 điểm/vòng** → judge đã chạm trần

Calibration là vòng lặp — AI Judge khớp dần qua từng vòng

Vòng 1

Người



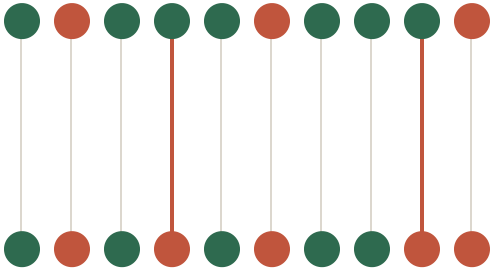
AI Judge

TPR **95%**
TNR **25%**

lệch 4/10

Vòng 2

Người



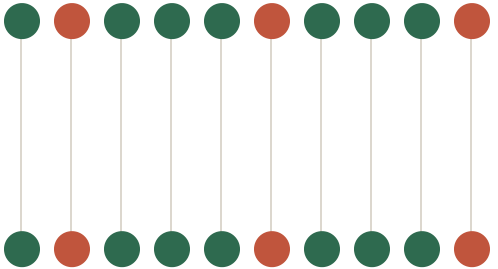
AI Judge

TPR **86%**
TNR **67%**

lệch 2/10

Vòng 3

Người



AI Judge

TPR **90%**
TNR **89%**

khớp — đạt ngưỡng ✓

Mỗi vòng: AI Judge chấm → so từng dòng với nhãn người → đọc chỗ lệch → sửa prompt (thêm ví dụ near-miss) → chấm lại. Dừng khi đạt ngưỡng PM đã chốt trước, rồi kiểm tra một lần trên tập test giữ riêng.

Calibrate judge không cần hạ tầng — một spreadsheet là đủ

calibration_round_3.xlsx

OUTPUT CỦA AGENT	JUDGE	EXPERT
Hoàn tiền đúng chính sách, giải thích rõ	✓	✓
Đúng chính sách nhưng bỏ sót câu hỏi thứ hai	✓	✗
Từ chối một yêu cầu hoàn toàn hợp lệ	✗	✗

▪ Dòng bất đồng — nơi duy nhất đáng đào sâu

1 Gửi expert 25–50 case, vài ngày một lần

Kèm cả verdict và lý do của judge để expert phản biện.

2 Chỉ đào sâu chỗ hai bên bất đồng

Bất đồng cho biết judge hiểu sai tiêu chí ở đâu — hoặc tiêu chí đang mơ hồ.

3 Sửa prompt của judge, đo lại vòng sau

Theo dõi tỉ lệ đồng thuận tăng dần qua từng vòng — ngay trong spreadsheet.

Tham chiếu: Hamel Husain — Your AI Product Needs Evals

Khi nào LLM Judge đã chạm trần?

Nhận ra thời điểm này quan trọng — tránh tốn thời gian tối ưu thứ không còn cải thiện được.

DẤU HIỆU 1

Model không thật sự hiểu domain

Reasoning strings cho thấy model thiếu nền tảng. Ví dụ: đánh giá legal citation hay medical recommendation — model không có năng lực ở domain đó.

Vấn đề là model capacity — không phải prompt.

DẤU HIỆU 2

Thêm prompt mà chất lượng gần như không nhích

Effort tăng nhưng improvement hầu như bằng không qua nhiều vòng. Đã rút gần hết giá trị có thể lấy từ model này.

Đã chạm điểm trần với model này.

DẤU HIỆU 3

Ngay cả humans cũng chưa thống nhất

Trên tiêu chí cảm tính, chính các expert chỉ đồng thuận với nhau ở mức ICC 0.07–0.18 — hệ số đồng thuận giữa người chấm, 0 = không khớp, 1 = tuyệt đối (Linde 2026). Khi chính người còn lệch nhau đến vậy, không thể kỳ vọng judge khớp hơn — chính tiêu chí đang hỏng.

Đo trần người-người trước — bất đồng cao giữa người là lỗi đặc tả rubric, không phải cơ hội tự động hoá.

Vấn đề không còn nằm ở cách viết prompt — mà ở giới hạn của chính model hoặc tiêu chí đang được đánh giá.

PART 07

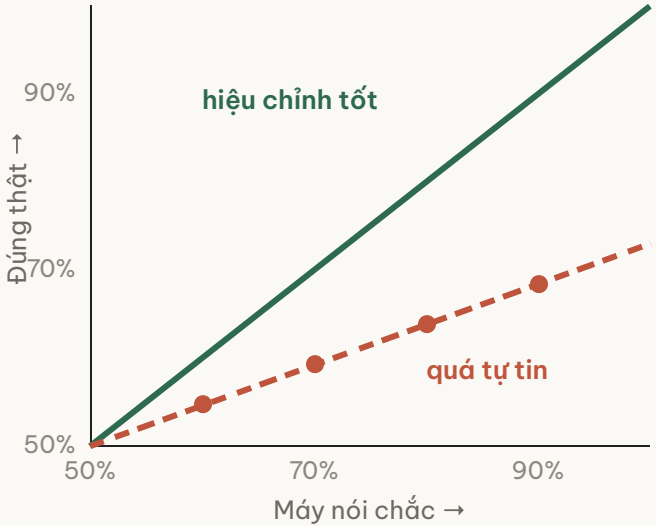
Đưa expert vào loop

Chia việc máy/người · Bằng chứng thay điểm số · Annotation tool · Label là tài sản

Câu hỏi đo lường ① và ④ — Đo cái gì, dụng cụ có đúng không?

Chỉ giao cho máy phần nó tự tin — khi độ tự tin đã được kiểm

① MÁY NÓI CHẮC X% CÓ ĐÚNG X%?



số minh hoạ

② NGƯỠNG SUY RA TỪ GIÁ HAI LOẠI SAI

Tự động khi chắc ≥ 1 − giá chuyển người / giá sai

Chuyển cho người xem	15.000đ
Một ticket xử lý sai	300.000đ
Ngưỡng tự động = 1 − 15/300	95%

BẢN CHẤT

Máy hiệu chỉnh tốt thì trong những lần nói chắc 90%, nó đúng khoảng 9/10. Lỗi dồn vào vùng kém tự tin — nên nhận phần tự tin, chuyển phần còn lại.

TRONG SẢN PHẨM CỦA BẠN

PRD mẫu ghi "độ tin < 0,7 → người xem". Con số 0,7 chỉ có nghĩa khi đã kiểm trên ticket thật của bạn.

Hai kiểm tra trước khi dùng độ tự tin để chia việc: **máy nói chắc X% có đúng X% không**, và **ngưỡng có suy ra từ giá hai loại sai không**. Thiếu một trong hai, 0,7 chỉ là con số đoán.

Chow (1970) · Guo et al. (2017), ICML · chi phí là ví dụ

Cho expert xem bằng chứng, đừng đưa sẵn điểm số

KHI MÁY GỢI Ý SẴN KẾT LUẬN

+2.9 điểm chính xác khi AI gợi ý đúng

−11.3 điểm khi AI gợi ý lệch — người bị kéo sai theo

Downside gấp **~4 lần** upside — điểm số gợi ý sẵn là can thiệp rủi ro cao.

Jabbour 2023 · RCT — thử nghiệm đối chứng ngẫu nhiên trên 457 clinician

MÁY XUẤT ĐIỂM SỐ

Tăng chi phí **bất đồng** — muốn phản bác phải tự dựng lập luận.

"Độ tin cậy: 0.86" → trượt

MÁY XUẤT BẰNG CHỨNG

Giảm chi phí **kiểm tra** — chỉ cần liếc đoạn được trích.

"Trích chính sách, mục 4.2: hoàn nếu trễ > 3 ngày" → đạt

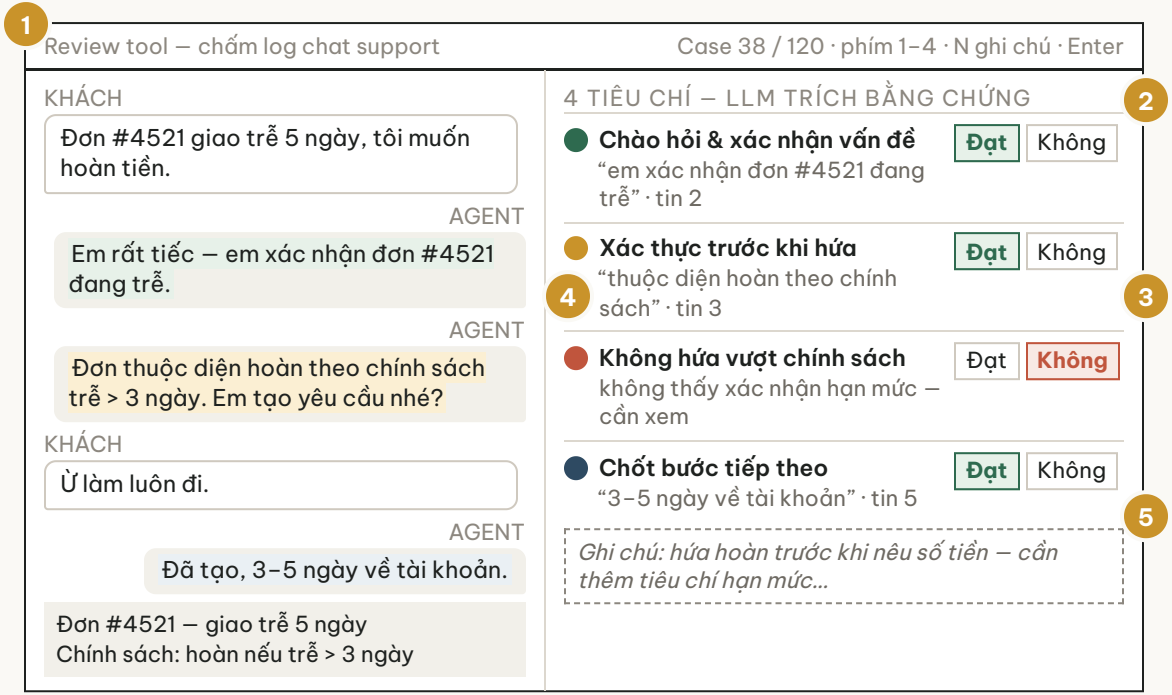
Phép thử thiết kế: **"Output này có giúp expert xác nhận hay bác bỏ nhanh hơn so với không có nó không?"**

THỨ TỰ MÀN HÌNH CHỐNG LỆCH Bằng chứng → Người phán đoán → Mới hiện điểm máy

Fogliato 2022

“Remove ALL friction” — một tool để expert chấm nhanh

Review một trace phải mở 4 tab → không ai review đủ nhiều → không ai biết hệ thống fail kiểu gì.

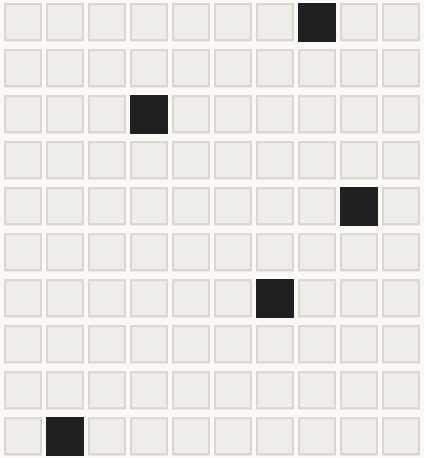


1. **Một màn hình đủ ngữ cảnh**
   trace, đơn hàng, chính sách — không mở tab
2. **Ngôn ngữ domain, giấu jargon AI**
   expert không cần biết trace, span, tool call
3. **Một quyết định nhị phân mỗi tiêu chí**
   Đạt / Không · phím tắt 1–4, Enter
4. **Bằng chứng trỏ đúng vị trí**
   màu tiêu chí = màu highlight trong log · máy không xuất điểm
5. **Ô ghi chú tự do**
   nguồn của trace code và tiêu chí mới

Build trong ~1 ngày — tool nội bộ nhỏ, không phải dự án hạ tầng.

Càng tự động, phần việc còn lại của người càng khó

TRƯỚC · NGƯỜI DUYỆT 100 TICKET / NGÀY



95 ca dễ giữ tay nghề mỗi ngày · 5 ca khó gặp khi còn tỉnh

SAU KHI TỰ ĐỘNG 95% · CÒN 5 TICKET



Chỉ còn ca khó và hiếm: **không còn ca dễ để luyện tay**, và phải canh một hàng đợi gần như không có gì xảy ra.

THIẾT KẾ CHO NGƯỜI

1. Cho expert chấm thêm một ít ca dễ, chọn ngẫu nhiên
2. Người phán trước — điểm của máy hiện sau
3. Phiên review ngắn

“Hệ thống điều khiển càng tiên tiến, phần đóng góp của người vận hành càng có thể trở nên quyết định.”
Lisanne Bainbridge (1983), “Ironies of Automation”, Automatica 19(6)

Tự động hoá càng giỏi, càng phải **thiết kế cho người**: giữ tay nghề, giữ phán đoán độc lập, giữ chú ý.

PART 08

Đọc kết quả & ra quyết định

Con số chỉ có giá trị khi nó dẫn tới một quyết định.

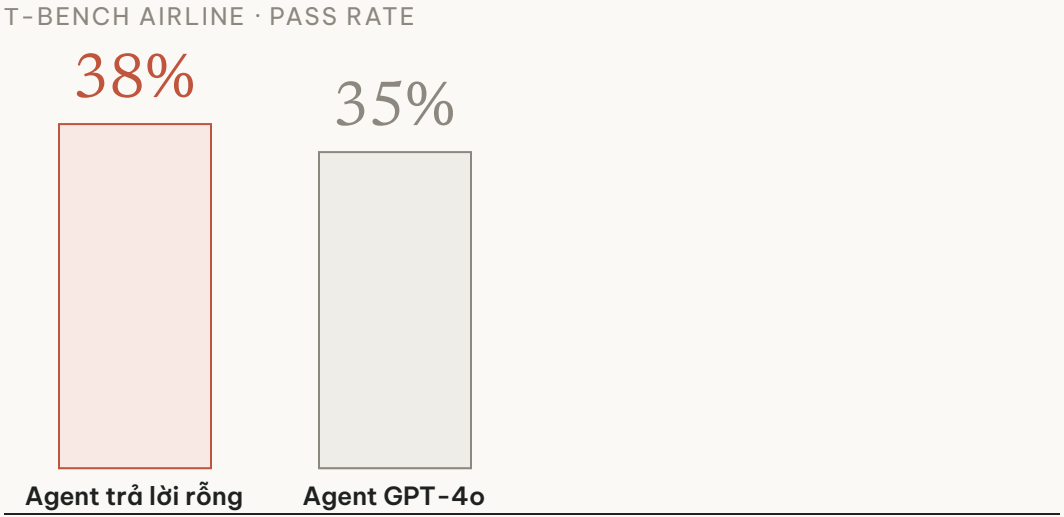
Câu hỏi mở bài — Đủ để ship chưa? Đủ để nói tốt hơn chưa?

Kiểm bài thi trước khi tin điểm

Một eval chỉ đáng tin khi cả hai mắt xích đều đúng.

Task validity
Task chỉ giải được khi agent có đúng năng lực cần đo — không có đường tắt.

Outcome validity
"Pass" thật sự nghĩa là việc đã thành công — không chỉ trông giống.



Không làm gì vẫn **cao điểm hơn** agent thật — vì bài thi coi "không đổi gì" là thành công.

CHẠY TRƯỚC AGENT THẬT | Agent rỗng | Luôn chọn nhãn phổ biến nhất | Luôn chuyển cho người

qua được check nào → check đó hỏng

Zhu et al. 2025 (ABC checklist) · Grace et al. 2026, Anthropic

CORE-Bench: 42% → 95% chỉ nhờ sửa lỗi chấm

Đọc kết quả eval trên dataset

Pass rate tổng là bao nhiêu?
So với phiên bản production

Failures tập trung ở đâu?
Nhóm input nào đang fail nhiều?

Lỗi có đi cùng nhau không?
Row nào fail nhiều evals cùng lúc?

Kết quả có bất thường không?
Pass rate = 0% hoặc quá thấp

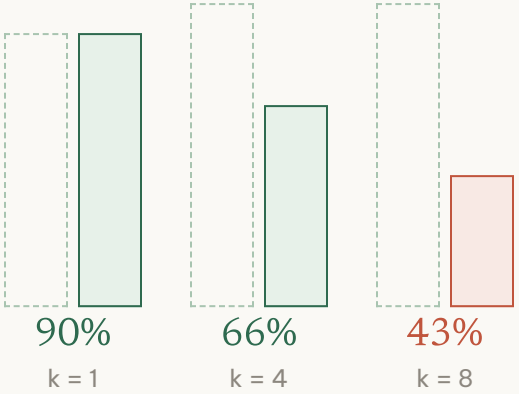
TÍN HIỆU TỐT	CẦN XEM LẠI
✓ Pass rate bằng hoặc cao hơn baseline	× Pass rate tụt đáng kể dù spot check trông ổn
✓ Failures rải đều — không nhóm nào nổi bật	× 80%+ failures từ cùng một loại input → failure mode cụ thể
✓ Failures phân tán, không row nào fail nhiều evals	× Cùng row fail 2+ evals → cụm input khó, không phải lỗi rời rạc
✓ Kết quả nằm trong range hợp lý, khớp expectation	**Pass rate = 0% → kiểm tra eval trước!** Regex sai, field sai — đừng vội điều tra agent.

Con số eval = agent × harness × grader. Điểm bất thường thì nghi cả ba.

Hai cái bẫy khi đọc kết quả: một lần chạy, và từng lượt

BẪY 1 · MỘT LẦN CHẠY

Agent đúng 90% mỗi lần. Tỉ lệ **đúng cả k lần** liên tiếp:



pass@k — khung đứt nét: đúng **ít nhất 1** trong k lần · **pass^k** — cột đặc: đúng **cả k** lần

Chỉ cần một lần đúng (sinh ý tưởng, code có test) → pass@k · Người dùng cần nhất quán (CSKH, đặt vé) → pass^k

Muốn đúng cả 8 lần với xác suất 90%: mỗi lần phải ≥ **98,7%**. Temperature 0 cũng không cố định — đo độ ổn định bằng cách chạy lại cùng case k lần.

BẪY 2 · CHẤM TỪNG LƯỢT

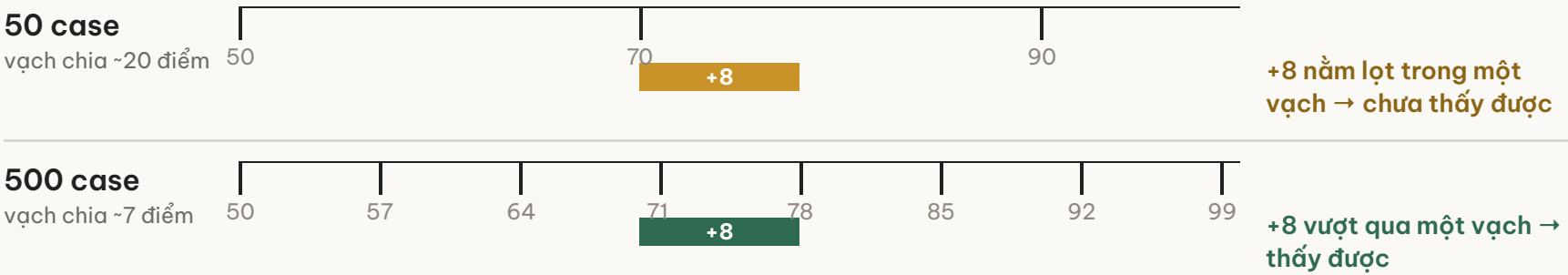
Lượt 1	Học viên: "Chỉ dùng tài liệu khoá mình nhé, đừng lấy nguồn ngoài."	
Lượt 2–4	AI tutor trả lời đúng tài liệu khoá	✓✓✓
Lượt 5	AI tutor trích một bài blog ngoài	✓

5/5 lượt Pass **· cả phiên Fail**

Với sản phẩm nhiều lượt, một test case là cả kịch bản + trạng thái đích.

Bộ test càng nhỏ, càng không nhìn thấy thay đổi nhỏ

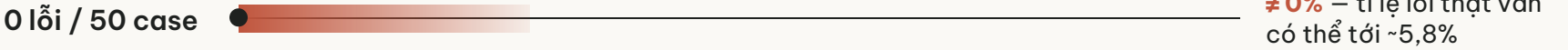
Như cái cân chỉ có vạch 1 kg: bạn tăng 200 g, kim không nhúc nhích. Cùng mức cải thiện +8 điểm, đo trên hai bộ test:



VẠCH CHIA THEO SỐ CASE

- **50** case → ~20 điểm
- **100** case → ~15 điểm
- **200** case → ~11 điểm
- **500** case → ~7 điểm

· muốn vạch nhỏ đi một nửa: gấp 4 lần số case

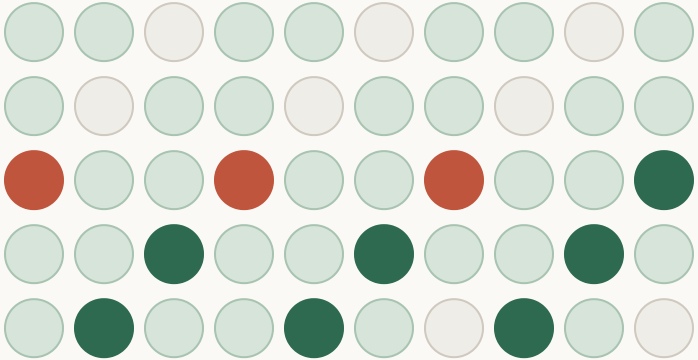


> “Chưa đủ dữ liệu để nói” là một kết luận hợp lệ.

So hai phiên bản trên cùng bộ test · độ tin 95%, power 80% · giả định ~30% case hai bản cho kết quả khác nhau.

Trả lời câu hỏi mở bài: 70% → 78% trên 50 ticket

50 TICKET · SO BẢN CŨ VỚI BẢN MỚI



● 32 cả hai đúng · ● 8 cả hai sai — không nói bản nào hơn

● 7 cải thiện · ● 3 regression — chỉ 10 ticket này mang thông tin

NẾU HAI BẢN NGANG NHAU, 10 TICKET ẤY NHƯ TUNG ĐỒNG XU

↑ ↑ ↑ ↑ ↑ ↑ ↑ ↓ ↓ ↓

Lệch **7–3** hoặc hơn xảy ra khoảng **1/3 số lần** — đồng xu thường cũng làm được.

1. **Đủ để ship? CÓ** — đạt ngưỡng 75% đã chốt trước
2. **Nói được là tốt hơn? CHƯA** — 7–3 chưa phân biệt được với may rủi

> Ngưỡng trả lời câu ①. Cái cân trả lời câu ②. Trong product review, nói "đạt ngưỡng ship" — không nói "tốt hơn 8%".

Bộ capability không cần 100% — pass rate là một quyết định sản phẩm

Cùng một con số 90% — là thành công với sản phẩm này, là thảm hoạ với sản phẩm khác.



NGƯỠNG PHỤ THUỘC | Chi phí của một lỗi | Có human review phía sau không | Ai là người chịu rủi ro

NỐI VÀO BUSINESS METRIC

Factual consistency **80%** → tự động hoá **30%** request · **90%** → **50%** · **98%** → **90%** — Chip Huyen, AI Engineering ch.4

Kèm **usefulness threshold**: dưới ngưỡng nào đó (vd 50%) thì vô dụng với mọi loại request — đó là sàn, không phải mục tiêu.

Mọi thay đổi đi qua cùng một cổng — và kết luận nào cũng kèm việc tiếp theo

CỔNG

1 Eval là release gate, không phải kiểm tra tuỳ hứng

Mỗi prompt change hoặc model upgrade chạy lại eval tự động. Smoke ~30 case mỗi commit · bộ đầy đủ hằng đêm · phần judge trước mỗi release.

2 Chốt ngưỡng trước khi iterate

Nhãn 90% · 0 lỗi bịa nghiêm trọng · latency 2s — viết ra trước.

"78% cũng ổn mà" — không phải threshold, là thương lượng.

3 Gate chỉ khoá bộ regression (~100%)

Bộ capability pass thấp là bình thường, không chặn ship. Giữ một holdout không dùng để sửa prompt.

Quy tắc ship: mọi check critical đạt ngưỡng, và không slice quan trọng nào tụt quá vạch chia của nó.

KẾT LUẬN → VIỆC TIẾP THEO

✓ Ship kèm kế hoạch monitoring tuần đầu

· sample 1–10% session production để chấm tiếp

· 3 tín hiệu drift: offline–online lệch dần · failure mode mới · feedback mâu thuẫn eval

· ngưỡng alert cụ thể — không phải "để ý xem sao"

⟳ Iterate đòn rẻ nhất trước

· 1 · Prompt — siết instruction, thêm near-miss

· 2 · Đổi model — chạy lại cả suite, kèm latency SLA

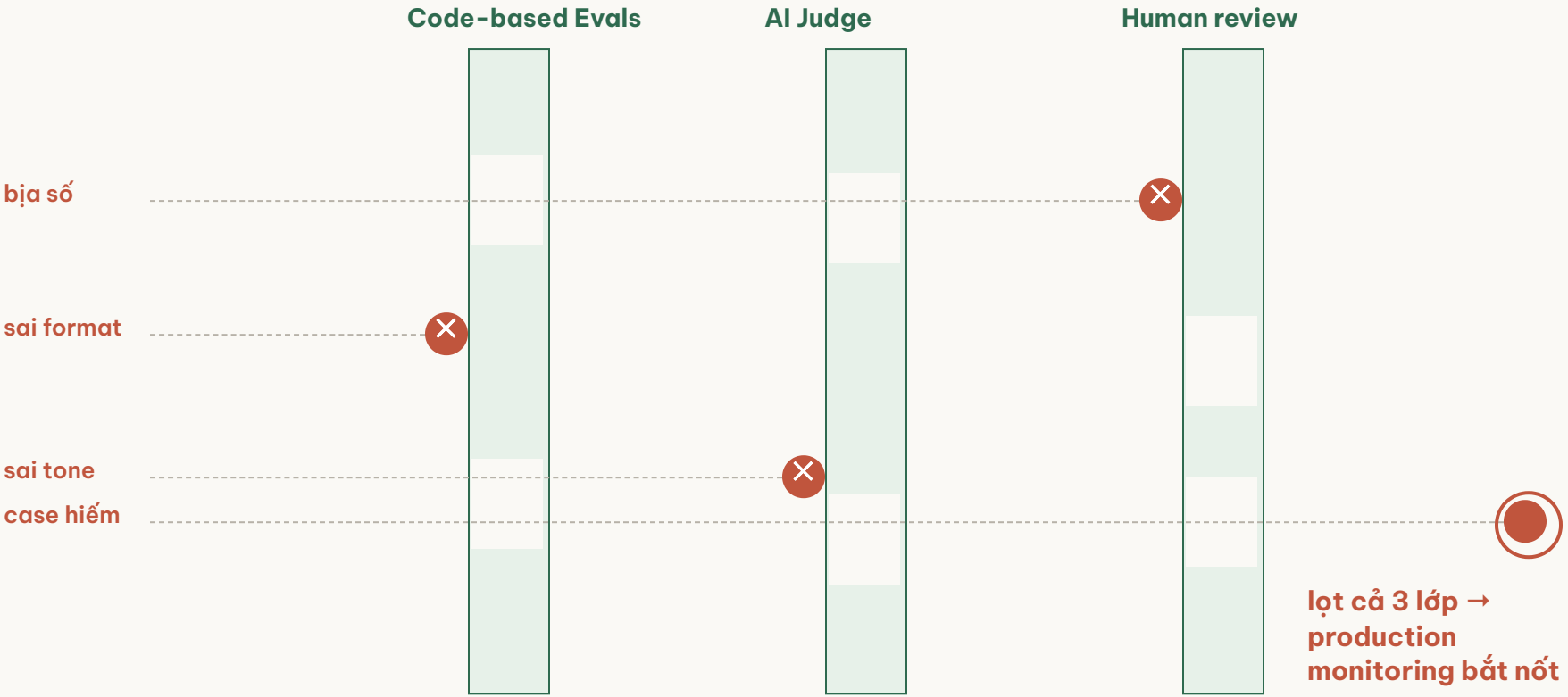
· 3 · Architecture — thêm tool, tách bước; đắt nhất, làm cuối

✕ Revert khi không có cải thiện đo được

· "cảm giác đáng lẽ giúp" mà số không nhích — không phải cải thiện

· quay về baseline, ghi vào experiment log · đừng giữ vì đã lỡ tốn công

Không lớp nào bắt được mọi lỗi — nên xếp lớp



Hệ thống evals hoàn chỉnh trông như thế này

1 Đọc trace
INPUT
Trace từ vibe check / production
OUTPUT
Trace codes — agent hỏng ở đâu

→

2 Dataset có chủ đích
INPUT
Trace codes + 3–5 dimension
OUTPUT
40–50 test input phủ đủ ô grid

→

3 Rubric nhị phân
INPUT
Trace codes
OUTPUT
Tiêu chí Pass/Fail hai người chấm ra cùng nhãn

→

4 Chia bộ chấm
INPUT
Từng tiêu chí
OUTPUT
Code · Judge · Human

→

5 Calibrate judge
INPUT
50–100 nhãn expert
OUTPUT
TPR/TNR đạt ngưỡng

→

6 Chạy & đọc report
INPUT
Mọi thay đổi
OUTPUT
Bảng report bên dưới ↓

Eval report — v1.4 so với v1.3		n = 200 · vạch chia ~11 điểm
Đúng format & nhãn chuẩn	Code	98% ✓ +2 · nhiễu
Không bịa ticket ID	Code	100% ✓
Bám nguồn	Judge TPR 88 / TNR 85	86% ✓ +2 · nhiễu
Empathy	Judge đã calibrate	79% → đọc trace −1 · nhiễu
Chuẩn chuyên môn	Human sample 10%	92% ✓
Coverage: **18/20 ô grid**		Ship có điều kiện · chưa nói v1.4 tốt hơn

Tổng kết

Đo công việc AI làm ra — định nghĩa rõ thế nào là "tốt"

- Đọc trace là user research mới — đừng tin lời agent nói, kiểm trạng thái cuối
- Thu hẹp scope đến khi viết được rubric nhị phân cả team chấm giống nhau
- Dataset: người cầm coverage, LLM chỉ diễn đạt — phủ độ khó, không phủ tần suất
- Đo đúng tầng — RAG chỉ là một tầng của hệ thống

Tự động hoá đúng thứ đáng tự động hoá

- Code là mặc định — judge cho sắc thái, và chỉ tin judge sau khi calibrate
- Judge dễ dãi hơn bạn tưởng — so với trần người-người, không so với 1.0
- Expert vào loop: bằng chứng thay điểm số, một nhãn dùng lại ba lần
- Bộ test là cái cân: chênh nhỏ hơn vạch chia thì chưa kết luận

AI Evals

Model mới ra sáng mai — team bạn có dám ship trong 24 giờ không?

Thực hành: 4 việc trên sản phẩm của nhóm

1 Đọc 20–30 trace thật

Trộn nguồn, giữ ≥30% ngẫu nhiên. Chưa có traffic: chạy 20 input theo grid. Ghi lỗi đầu tiên theo 4 quy tắc ghi chú.

2 Gom thành 3–5 trace codes

Đếm, xếp ưu tiên theo tần suất × mức nặng. Trọng số do nhóm chốt và ghi ra.

3 Viết 1 evaluator và đo nó

Code đứng đầu → code check (nếu có referent rõ) hoặc 1 judge Pass/Fail kèm 2 ví dụ sát ranh giới. Gán nhãn tay ≥20 case, báo riêng tỉ lệ nhận đúng output tốt và tỉ lệ bắt được output lỗi.

4 Gộp thành golden set v0.1

Có trường `slice`, commit vào repo cùng code.

Tài liệu tham khảo

Hamel Husain & Shreya Shankar — “Advanced evals: How to find (and fix) hidden AI failures in your product”, Lenny’s Newsletter, 2026

Hamel Husain — “A Field Guide to Rapidly Improving AI Products”

Anthropic — “Demystifying evals for AI agents”

Sarah Sachs (Notion) & Carlos Esteban (Braintrust) — “How to build world-class AI products”

Zheng et al. — “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena”, NeurIPS 2023

Jabbour et al. — AI và chẩn đoán bệnh nhân nội trú, JAMA 2023

Hamel Husain — “Your AI Product Needs Evals”

Shankar et al. — “Who Validates the Validators?”, UIST 2024

Chip Huyen — AI Engineering, ch.4 “Evaluate AI Systems”

RAGAS — tài liệu các chỉ số

Si, Yang & Hashimoto — “Can LLMs Generate Novel Research Ideas?”, 2024

Fogliato et al. — “Who Goes First?”, FAccT 2022

Tư liệu bài giảng Ngày 14 của Trần Quang Thiện